

Relatório do Trabalho Final: Sistema de Elevadores

João Bruno dos Santos, João Lucas Medina Kormann

Universidade Federal de Santa Catarina

I. DESCRIÇÃO DO PROJETO

Este documento apresenta a entrega técnica do sistema de controle lógico desenvolvido para o gerenciamento automatizado de dois elevadores, projetado especificamente para atender uma planta de oito pavimentos com alto desempenho e confiabilidade. A solução foi estruturada utilizando a linguagem Assembly para a arquitetura MIPS.

A validação e a interação com o sistema ocorrem através do simulador MARS (MIPS Assembler and Runtime Simulator), integrando-se especificamente à ferramenta Digital Lab Sim. Esta ferramenta fornece a interface visual necessária para emular os componentes físicos do sistema, incluindo um teclado matricial para a solicitação de chamadas (subida e descida) e displays de sete segmentos que indicam, em tempo real, o andar atual de cada um dos dois elevadores.

A premissa central desta implementação é oferecer uma gestão inteligente de tráfego que priorize a redução do consumo energético e a otimização do tempo de espera do usuário. Diferente de controladores simples que operam de forma sequencial, o sistema entregue conta com um algoritmo de designação de tarefas (Scheduler) capaz de tomar decisões estratégicas em tempo real. O objetivo é maximizar a utilização dos recursos, mantendo, sempre que possível, apenas um elevador em operação para atender demandas isoladas, ativando a segunda unidade apenas quando estritamente necessário para garantir o fluxo contínuo.

Para garantir a melhor experiência de uso, a lógica de controle incorpora um mecanismo de proximidade: ao receber uma solicitação de chamada — seja para subir ou descer — o sistema avalia instantaneamente qual das cabines está mais próxima do andar solicitante, despachando o equipamento que pode atender o chamado no menor tempo possível. Além disso, a solução respeita

rigorosamente os parâmetros temporais de deslocamento, calibrados para simular um trânsito de quatro segundos entre pavimentos, proporcionando uma simulação realista da física de movimentação do equipamento.

Do ponto de vista da engenharia de software, a solução foi construída com uma arquitetura orientada a componentes. Isso significa que o sistema é modular, com blocos dedicados para o tratamento de entradas e saídas (interface com o usuário), gerenciamento de fila de requisições e controle de motores. Essa abordagem assegura não apenas a estabilidade da operação atual, mas facilita futuras manutenções e expansões do sistema sem comprometer o núcleo lógico de funcionamento. A interface visual fornece feedback imediato através de displays numéricos, garantindo que os usuários tenham visibilidade constante sobre a localização atual de cada elevador.

II. DESCRIÇÃO FUNCIONAL E GUIA DE OPERAÇÃO

Esta seção detalha os procedimentos operacionais para a interação com o sistema de controle de elevadores. A interface homem-máquina foi configurada utilizando a ferramenta de simulação Digital Lab Sim, que centraliza tanto os comandos de entrada (painel de chamadas) quanto o feedback visual de status (indicadores de andar).

A. INTERFACE DE CONTROLE

A operação do sistema é realizada através de um teclado matricial numérico e alfanumérico, mimetizando o painel de controle físico de uma edificação. A visualização do posicionamento das cabines é feita por meio de dois displays de sete segmentos.

Displays de Status: O sistema possui dois indicadores visuais. O display da esquerda monitora em tempo real a posição do Elevador 1, enquanto o display da direita reflete a posição do Elevador 2.

Teclado de Comando: Utilizado para inserir as variáveis da solicitação (origem, sentido e destino).

B. PROCEDIMENTO DE SOLICITAÇÃO DE SERVIÇO

Para garantir o despacho correto da cabine e a otimização do tráfego, o sistema requer uma sequência lógica de entrada de dados. O operador ou usuário deve seguir o fluxo de 4 etapas descrito abaixo no teclado do simulador:

1. Identificação do Andar de Origem (Onde o usuário está): Pressione a tecla numérica correspondente ao andar atual, variando de 0 (Térreo) a 7 (Sétimo Andar).

2. Definição de Sentido (Para onde o usuário quer ir): O sistema distingue entre chamadas de subida e descida para otimizar a rota.

- Utilize a tecla c para solicitar Subida.
- Utilize a tecla b para solicitar Descida.

3. Definição do Andar de Destino (Onde o usuário vai descer): Insira novamente uma tecla numérica (0 a 7) indicando o pavimento final da viagem.

4. Confirmação e Envio (Enter): Para processar a solicitação e acionar os motores, pressione a tecla f para confirmar o comando.

Nota: O sistema possui uma validação interna que ignora comandos fora da sequência lógica ou teclas inválidas, prevenindo comportamentos erráticos das cabines.

C. COMPORTAMENTO DO SISTEMA E FEEDBACK

Após a confirmação da solicitação (tecla f), o Controlador Lógico (Scheduler) assume a operação:

Alocação Inteligente: O sistema calcula instantaneamente a distância de ambas as cabines em relação ao andar de origem. A requisição será atendida pelo elevador que estiver mais próximo e disponível, visando a economia de energia e tempo.

Tempo de Deslocamento: O movimento entre andares é simulado com precisão temporal. O display atualizará o número do andar a cada 4

segundos, representando o tempo físico de deslocamento da cabine entre os pavimentos.

Prioridade de Uso: Em cenários onde um elevador já está em movimento e o outro está parado, o sistema priorizará a movimentação daquele que já está ativo, desde que a rota seja compatível, mantendo o segundo elevador em stand-by para conservação de recursos.

III. ESPECIFICAÇÕES TÉCNICAS E ARQUITETURA

O sistema foi desenvolvido sobre a arquitetura MIPS, utilizando o simulador MARS. O software opera sob um laço principal que orquestra sequencialmente a sondagem de periféricos, o processamento lógico e a atualização de saídas. A ausência de um Sistema Operacional implica que todo o gerenciamento de recursos, tempo e memória é realizado diretamente via instruções de Assembly.

A. ORGANOGRAMA DA ARQUITETURA E FLUXO DE CONTROLE

A execução do sistema é centralizada no procedimento main, que atua como o kernel da aplicação. O fluxo de controle obedece à seguinte hierarquia sequencial a cada iteração do ciclo:

IOHandler_process: Varredura de hardware (Teclado).

Scheduler_process: Decisão e despacho de tarefas.

Elevator1_process: Máquina de estados do Elevador 1.

Elevator2_process: Máquina de estados do Elevador 2.

Display Update: Escrita direta nos endereços de memória dos displays.

Syscall Delay: Sincronização temporal do loop.

B. ESTRUTURAS DE DADOS E GERENCIAMENTO DE MEMÓRIA

A organização dos dados segue o padrão de structs definidas na seção .data, permitindo o acesso indexado aos atributos de cada elevador.

1. Estrutura do Elevador (Elevator Struct) Cada elevador possui um bloco de memória contíguo dedicado, contendo:

- _andarAtual (Word): Armazena a posição física (0-7).
- _andarDestino (Word): O setpoint para onde o motor deve levar a cabine.

- `_emMovimento` (Word): Flag booleana (0/1) indicando estado do motor.
- `_passoTimer` (Word): Acumulador para controle de temporização de deslocamento.
- `_request` (Array de 3 Words): Buffer local que recebe a ordem do Scheduler (Origem, Sentido, Destino).

2. Fila de Entrada (InputQueue) Para mitigar a assincronicidade entre a digitação do usuário e o ciclo de processamento, foi implementada uma estrutura de buffer linear de 4 palavras (inputQueue), mapeada em Constants.asm. Ela funciona como uma máquina de estados de preenchimento progressivo:

- Offset 0x00 (Origem): Aceita valores 0-7.
- Offset 0x04 (Sentido): Aceita 0x0B (Descer) ou 0x0C (Subir).
- Offset 0x08 (Destino): Aceita valores 0-7.
- Offset 0x0C (Confirmação): Aceita 0x0F (Tecla 'f'). O Scheduler só consome esta fila quando o offset 0x0C é preenchido, garantindo atomicidade na transação.

C. SUBSISTEMA DE ENTRADA E SAÍDA

1. Leitura do Teclado Matricial (IOHandler.asm) A interação utiliza E/S Mapeada em Memória (MMIO). O procedimento de leitura não utiliza interrupções de hardware, mas sim uma técnica de Polling (varredura) matricial:

Endereçamento: Utiliza o endereço base 0xFFFF0012 para habilitar linhas e 0xFFFF0014 para ler colunas.

Algoritmo de Varredura: O código itera um bitmask (1, 2, 4, 8) nas linhas da matriz. Ao detectar um bit ativo na leitura, calcula-se o valor da tecla baseada na interseção Linha x Coluna usando a keyTable.

2. Atualização dos Displays A renderização visual é realizada pela escrita direta de bytes nos endereços 0xFFFF0010 (Elevador 1 - Direita) e 0xFFFF0011 (Elevador 2 - Esquerda). O sistema utiliza uma tabela de conversão (displayTable) que mapeia o valor inteiro do andar (ex: 3) para o código hexadecimal correspondente aos segmentos do display (ex: 0x4F).

D. TEMPORIZAÇÃO E SINCRONIZAÇÃO (TIMERS)

Clock do Sistema: O procedimento DELAY_PROC utiliza a syscall 30 (System Time) para capturar o timestamp atual e criar um atraso bloqueante, definindo o heartbeat do sistema.

Temporizador de Deslocamento: Dentro da lógica de cada elevador (MoveLogic), existe um acumulador chamado passaAndar. A cada ciclo do loop principal, incrementa-se no ElevatorX_passoTimer. Quando este acumulador atinge 4000 (4 segundos), o sistema executa a transição de andar e reseta o acumulador. Isso simula a física de velocidade constante independente da velocidade de processamento da CPU.

IV. CONCLUSÃO E CONSIDERAÇÕES FINAIS

O projeto entregue cumpre com êxito os requisitos funcionais estabelecidos para a automação do sistema de transporte vertical do edifício. A solução implementada em linguagem de baixo nível (Assembly MIPS) demonstrou robustez no controle e precisão no gerenciamento temporal, respeitando o intervalo mandatório de 4 segundos entre andares. A lógica de escalonamento (Scheduler) provou ser eficaz em sua função primária: reduzir o deslocamento desnecessário das cabines, priorizando o elevador mais próximo e economizando energia operacional conforme solicitado.

No entanto, é necessário destacar as limitações inerentes à versão atual e os pontos de atenção identificados durante os testes de validação:

Complexidade da Interface de Entrada (Usabilidade): A solução atual, limitada pela interface de simulação Digital Lab Sim, exige que o usuário insira manualmente uma sequência de quatro dados para processar uma chamada (Andar Atual -> Sentido -> Destino -> Confirmação). Em um cenário de alta demanda, este processo é suscetível a erros humanos de operação e é consideravelmente mais lento do que o sistema padrão de "chamada por botão único".

Capacidade de Fila de Requisições: O módulo de gerenciamento de memória (inputQueue) foi dimensionado para processar uma transação completa por vez. O sistema não possui um buffer robusto para armazenar múltiplas chamadas simultâneas.